

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

**SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY**

GRADUATION RESEARCH REPORT (GR1)

**Topic: Design and Development of a Web-Based Fitness Platform for
Beginner Gym Users**

Project Code Name: FiFB (Fitness for Beginners)

NGÔ ĐỨC HUY

Huy.nd235946@sis.hust.edu.vn

Program: Global ICT

Supervisor: Professor Cao Tuan Dung

Department:

School: School of Information and Communications Technology

HANOI, 07/2026

Table of Contents

Chapter 1 – Introduction.....	4
1.1 Scientific and Technological Context	4
1.2 Problem Significance and Motivation.....	5
1.3 Objectives and Scope	5
1.4 Research Tasks	6
1.5 Planned Roadmap: GR1 → GR2	7
Chapter 2 – Related Work and Practical Survey.....	7
2.1 Overview of Existing Fitness Platforms	7
2.2 Comparative Analysis.....	7
2.3 Derived Requirements for the Foundation Demo.....	8
2.4 GR1 Feature Scope and Exclusions	9
Chapter 3 – Theoretical Foundations and Technology Stack	9
3.1 Client-Server Architecture and RESTful APIs.....	9
3.2 Authentication and Authorization	10
3.3 Role-Based Access Control and Actor Separation	11
3.4 Relational Database Design.....	11
3.5 GR1 Scope Boundary and Demo Focus	11
3.6 Technology and Framework Selection.....	12
Chapter 4 – FiFB Product	13
4.1 Functional Requirements.....	13
4.2 Non-Functional Requirements	15
4.3 System Architecture.....	15
4.4 Database Design	16
4.5 Backend Implementation.....	18
4.5.1 API Endpoints.....	18
4.5.2 Middleware Pipeline.....	18
4.5.3 Exercise and Equipment Data Management	19
4.5.4 Core Role and Authorization Flow	19
4.6 Frontend Implementation	19
4.6.1 Application Structure.....	19
4.6.2 Routing and Access Control.....	20
4.6.3 HTTP Client and Token Management	20

4.6.4 Role State Management.....	20
4.6.5 Shared Components Used in the Core Demo	20
4.7 Role-Based Demo Workflows	21
4.7.1 Core Actions by Role.....	21
4.7.2 User Exercise Lookup Flow	21
4.8 Scope Boundary for Additional Implementations	21
4.8.1 Implemented Extensions Not Evaluated in GR1	21
4.8.2 Reason for Exclusion	22
4.8.3 Future Use of Existing Code	22
<i>Chapter 5 – Results and Evaluation.....</i>	<i>22</i>
5.1 Completion Status.....	22
5.2 Demonstration and Outputs	23
5.3 Functional Evaluation.....	26
5.3.1 Authentication and Role Access	26
5.3.2 User Management.....	27
5.3.3 Exercise and Equipment Management.....	27
5.3.4 User Exercise Lookup	27
5.3.5 Scope Assessment.....	27
5.4 Limitations	27
5.5 Development Plan for GR2	28
<i>References</i>	<i>28</i>
<i>Appendices</i>	<i>29</i>

LIST OF ABBREVIATIONS

Abbreviation	Full Term
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ER	Entity–Relationship
GR1	Graduation Research 1
GR2	Graduation Research 2
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
MVP	Minimum Viable Product
OTP	One-Time Password
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier

Chapter 1 – Introduction

1.1 Scientific and Technological Context

Digital fitness platforms can help beginner gym users access consistent exercise information through a centralized web application. For a GR1 project, the main academic value is not the number of advanced features, but the ability to translate a practical problem into clear requirements, user roles, a relational database design, and a working full-stack demonstration.

FiFB is therefore developed as a client–server web system using React for the frontend, Express.js for the backend, and PostgreSQL for persistent data. These technologies are sufficient to demonstrate authentication, role-based access, user management, exercise management, equipment classification, and exercise lookup without requiring advanced real-time or AI components.

The project applies core software engineering knowledge from requirements analysis, system architecture, database design, API development, frontend development, and basic testing within the Information Technology curriculum at HUST.

1.2 Problem Significance and Motivation

Beginner gym users and system administrators face several basic problems that motivate the FiFB foundation demo:

1. Information fragmentation: Exercise names, instructions, target muscles, and required equipment are distributed across many sources, making reliable lookup difficult for beginners.
2. Difficulty selecting suitable exercises: Beginners need a simple way to search and filter exercises by body part, muscle group, and equipment or machine category.
3. Lack of centralized management: A web platform needs clearly separated Admin, Trainer, and User roles so that accounts and exercise data can be maintained consistently.

FiFB addresses these core problems by providing a centralized exercise library, role-based interfaces, user-account administration, exercise-content administration, equipment taxonomy management, and a searchable exercise interface for users.

1.3 Objectives and Scope

Objectives

The primary objective of GR1 is to design and build a foundation demo of the FiFB web platform with the following capabilities:

- Allow users to browse, search, filter, and view detailed exercise information, including body parts, target muscles, and required equipment.
- Provide secure login and role-based access for the three system roles: User, Trainer, and Admin.
- Allow Admin to list users, inspect user information, and change account status.
- Allow Admin to create, update, review, and deactivate exercise records.
- Allow Admin to list and add exercise equipment or machine categories, and associate them with exercises.
- Allow Trainer to maintain a basic trainer profile and submit exercise content for Admin review.

A secondary objective is to organize the code into a modular architecture that can be extended later, while keeping the GR1 evaluation focused on the required foundation functions.

Scope

The evaluated scope of GR1 includes:

- Authentication and authorization based on JWT.
- Role definition and access control for User, Trainer, and Admin.
- Relational database design for users, trainer profiles, exercises, body parts, muscles, and equipment categories.
- Basic Trainer profile management and exercise submission for review.
- Admin user management, including list, filter, detail, and status update operations.
- Admin exercise management, including creation, update, review, and deactivation.
- Equipment or machine-category management through the exercise taxonomy module.
- User exercise lookup through keyword search, filters, pagination, and exercise-detail pages.

Although the repository contains additional implemented modules, the following are outside the evaluated GR1 scope of this report: favorite exercises, workout plans and workout sessions, statistics and progression, trainer certificates and connections, reviews, chat, notifications, Redis caching, deployment automation, online payment, marketplace functions, and AI features.

1.4 Research Tasks

The following tasks were defined for the focused GR1 scope:

1. Problem and requirement analysis: Identify the needs of beginner users and define the responsibilities of User, Trainer, and Admin.
2. System design: Design the client–server architecture, role-based access model, database schema, API groups, and primary user interfaces.
3. Database design: Model users, trainer profiles, exercises, equipment, body parts, muscles, and their many-to-many relationships.
4. Backend development: Implement the Express.js modules for authentication, users, trainers, exercises, exercise taxonomy, and core Admin operations.
5. Frontend development: Implement login, role-protected routing, Admin user and exercise pages, Trainer submission pages, and User exercise lookup pages.
6. Verification: Manually test the main role flows and use existing linting and unit-test support to check the codebase.

1.5 Planned Roadmap: GR1 → GR2

Phase	Period	Key Deliverables
GR1	2025.2	Foundation demo: role model, database design, login, user management, exercise management, equipment taxonomy, and User exercise lookup.
GR2	2026.1	Complete physical machine inventory management, improve Trainer workflows, expand testing and UI/UX, and selectively integrate advanced features.

This roadmap deliberately separates the foundation requirements from optional extensions. GR1 demonstrates that the core architecture, database, and role-based functions work; GR2 can then extend the same modules without redefining the entire system.

Chapter 2 – Related Work and Practical Survey

2.1 Overview of Existing Fitness Platforms

Three representative platforms and data sources were reviewed to derive requirements for the FiFB foundation demo:

1. Jefit (jefit.com): A fitness application with a large exercise library and filtering functions. It provides a useful reference for how exercise information can be presented to users.

2. Trainerize (trainerize.com): A trainer-oriented platform that demonstrates the value of separating trainer and client responsibilities, although its scope is broader than the FiFB GR1 demo.
3. ExerciseDB (exercisedb.io): An exercise-data API containing GIF demonstrations, target muscles, body parts, and equipment information. It is used as a reference data source rather than as a complete end-user application.

2.2 Comparative Analysis

[Bảng 2.1: Comparison of Existing Fitness Platforms]

Feature	Jefit	Hevy	Trainerize	ExerciseDB API	FiFB (GR1)
Exercise Library with GIF	✓	Partial	✓	✓ (API only)	✓
Taxonomy-based Filtering	Partial	✗	Partial	✓	✓
Role-based access control	Partial	Partial	✓	✗	✓
User account management	Partial	Partial	✓	✗	✓
Exercise content management	Partial	Partial	✓	API only	✓
Equipment category management	Partial	Partial	Partial	✓	✓
Trainer exercise contribution	✗	✗	✓	✗	✓
Admin Moderation	✗	✗	✓	✗	✓
Open / Web-based	Partial	Partial	✓	API only	✓
Beginner-focused	Partial	✗	✗	✗	✓

The comparison leads to three findings relevant to GR1:

- Exercise libraries are useful only when users can search and filter structured data such as body parts, muscles, and equipment.
- Trainer-oriented systems show the importance of role separation, but GR1 only needs a basic Trainer role rather than a complete trainer–client ecosystem.
- ExerciseDB is suitable as an initial data source, while FiFB must still provide its own database, management interface, authorization rules, and user-facing web pages.

2.3 Derived Requirements for the Foundation Demo

Based on the survey and the GR1 assignment, the following requirements were selected:

Core functional requirements:

- A standardized exercise library with name, GIF, instructions, body-part, muscle, and equipment metadata.
- Keyword search, taxonomy-based filtering, pagination, and exercise-detail viewing for User.
- Login and role-based authorization for User, Trainer, and Admin.
- Admin functions for listing users and changing account status.
- Admin functions for creating, updating, reviewing, and deactivating exercises.
- Equipment or machine-category management and exercise–equipment association.
- Basic Trainer profile management and exercise submission for Admin review.
- A relational database schema that preserves referential integrity between users, exercises, and taxonomy data.
- A clear separation between the core GR1 requirements and optional extensions already present in the repository.

Quality requirements:

- Reasonable response time for standard login, CRUD, search, and filtering operations in a demonstration environment.
- Role-based access control so that each role can only use permitted functions.
- Foreign keys, status constraints, and normalized many-to-many mappings for data integrity.
- A modular backend structure that separates routes, controllers, services, repositories, and validation.
- Secure password storage and token-based authentication.

2.4 GR1 Feature Scope and Exclusions

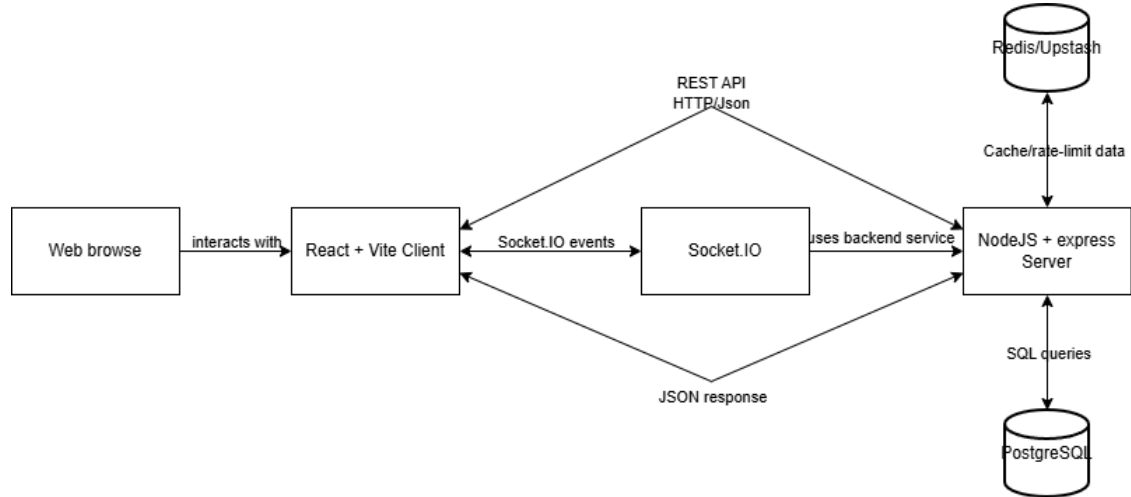
The GR1 report evaluates only the foundation functions required by the assignment: system design, role definition, database design, login, user management, exercise management, equipment taxonomy management, and User exercise lookup. This focused scope is sufficient to demonstrate a working platform foundation.

Additional implemented code for workout tracking, trainer connections, certificates, reviews, chat, notifications, caching, CI/CD, deployment, and future AI functions is not used as evidence of GR1 completion and is not described in detail.

Chapter 3 – Theoretical Foundations and Technology Stack

3.1 Client–Server Architecture and RESTful APIs

FiFB follows the client–server architectural pattern, where the frontend (client) and backend (server) are developed as separate applications communicating over HTTP.



[Figure 3.1: Client–Server Architecture Overview. The image reflects the extended repository; the evaluated GRI path is React Client → Express API → PostgreSQL.]

The backend exposes a RESTful API following standard HTTP semantics:

- Resource-based URLs, for example ``/api/exercises``, ``/api/users``, and ``/api/exercise-taxonomy/equipments``.
- Standard HTTP methods: ``GET`` for retrieval, ``POST`` for creation, ``PATCH`` for partial updates, ``DELETE`` for removal.
- Consistent response format: All responses follow a standardized JSON envelope: ``{ "success": true/false, "data": {...}, "message": "..."}``.
- Pagination: List endpoints return paginated results with ``items``, ``page``, ``limit``, ``total``, and ``totalPages`` fields.
- Error format: Errors include machine-readable ``code``, human-readable ``message``, and optional ``details`` for validation errors.

The separation of concerns between client and server allows independent scaling, testing, and potential replacement of either tier.

3.2 Authentication and Authorization

FiFB uses JWT-based authentication so that the backend can identify the current account and enforce role-specific permissions.

The core design decisions are:

- After login, the backend issues an access token containing the user identifier, role, and account status. The frontend sends this token in the `Authorization: Bearer` header for protected requests.
- A refresh-token mechanism supports session continuity; password hashes are stored with bcrypt rather than as plain text.
- Password hashing: bcrypt is used to protect stored passwords.
- Account verification: registration can place accounts in a pending state until email verification is completed.

Authorization is enforced at three levels:

1. Middleware layer: `authenticate` validates the token, while role middleware checks whether the request is made by User, Trainer, or Admin.
2. Service layer: business rules verify that only permitted roles can manage users, exercises, trainer data, or taxonomy data.
3. Database layer: foreign keys, unique constraints, and valid status values protect data consistency.

3.3 Role-Based Access Control and Actor Separation

Role-based access control is central to the GR1 demo because the same web platform serves three actors with different responsibilities:

- User: logs in, manages a basic profile, searches and filters exercises, and views exercise details.
- Trainer: has User-level exercise access, manages a trainer profile, and can submit new exercise content for review.

Admin: manages user status, exercise records, submitted exercises, and exercise taxonomy such as equipment, body parts, and muscles.

3.4 Relational Database Design

PostgreSQL is used because the GR1 domain contains strongly related data: accounts have roles, exercises are associated with multiple body parts, equipment categories, and muscles, and Trainer profiles extend user accounts.

Key database design principles applied in FiFB:

- Normalization: body parts, equipment, and muscles are stored separately from exercises and connected by junction tables, reducing duplication and supporting consistent filters.
- Status-based lifecycle: users and exercises use status values so records can be locked, disabled, inactive, or rejected without immediately losing historical data.

- **UUID primary keys:** database records use UUID identifiers to avoid predictable sequential identifiers and simplify relationships across modules.
- **Constraints:** role, status, and relationship constraints prevent invalid data from being stored.
- **JSONB instructions:** ordered exercise instruction steps can be stored without creating an unnecessarily complex table structure.
- **Indexes:** frequently queried fields such as user email, user status, exercise status, exercise name, and taxonomy identifiers are indexed for lookup operations.

3.5 GR1 Scope Boundary and Demo Focus

A clear scope boundary is necessary because the repository has grown beyond the minimum GR1 assignment.

1. The core evaluation follows only the path required to demonstrate the platform foundation: authentication, role checking, user administration, exercise administration, equipment taxonomy, and User exercise lookup.
 2. Modules for favorites, workout plans, workout sessions, trainer connections, reviews, chat, notifications, caching, and deployment remain in the source code as extensions.
- These extensions are intentionally excluded from the detailed report so that they do not obscure the required analysis and do not create the impression that GR1 success depends on advanced features.
 - This approach also makes the limitations of the current equipment implementation explicit: it is an exercise-equipment taxonomy, not yet a full physical machine inventory system.

3.6 Technology and Framework Selection

[Bảng 3.1: Technology Stack Summary]

Layer	Technology	Purpose
Frontend	React	UI component library
	Vite	Build tool and dev server
	React Router DOM	Client-side routing
	Axios	HTTP client
	Tailwind CSS	Utility-first CSS framework
	Lucide React	Icon library
	React Hook Form	Form state management
	Node.js	JavaScript runtime
Backend	Express.js	HTTP framework
	PostgreSQL	Relational database
	jsonwebtoken	JWT creation and verification
	bcryptjs	Password hashing
	Zod	Schema validation
	Helmet	HTTP security headers

	pg (node-postgres)	PostgreSQL driver
	ESLint	Code linting
	Prettier	Code formatting
	Jest	Unit testing
	Supertest	HTTP integration testing
	Nodemon	Development auto-restart

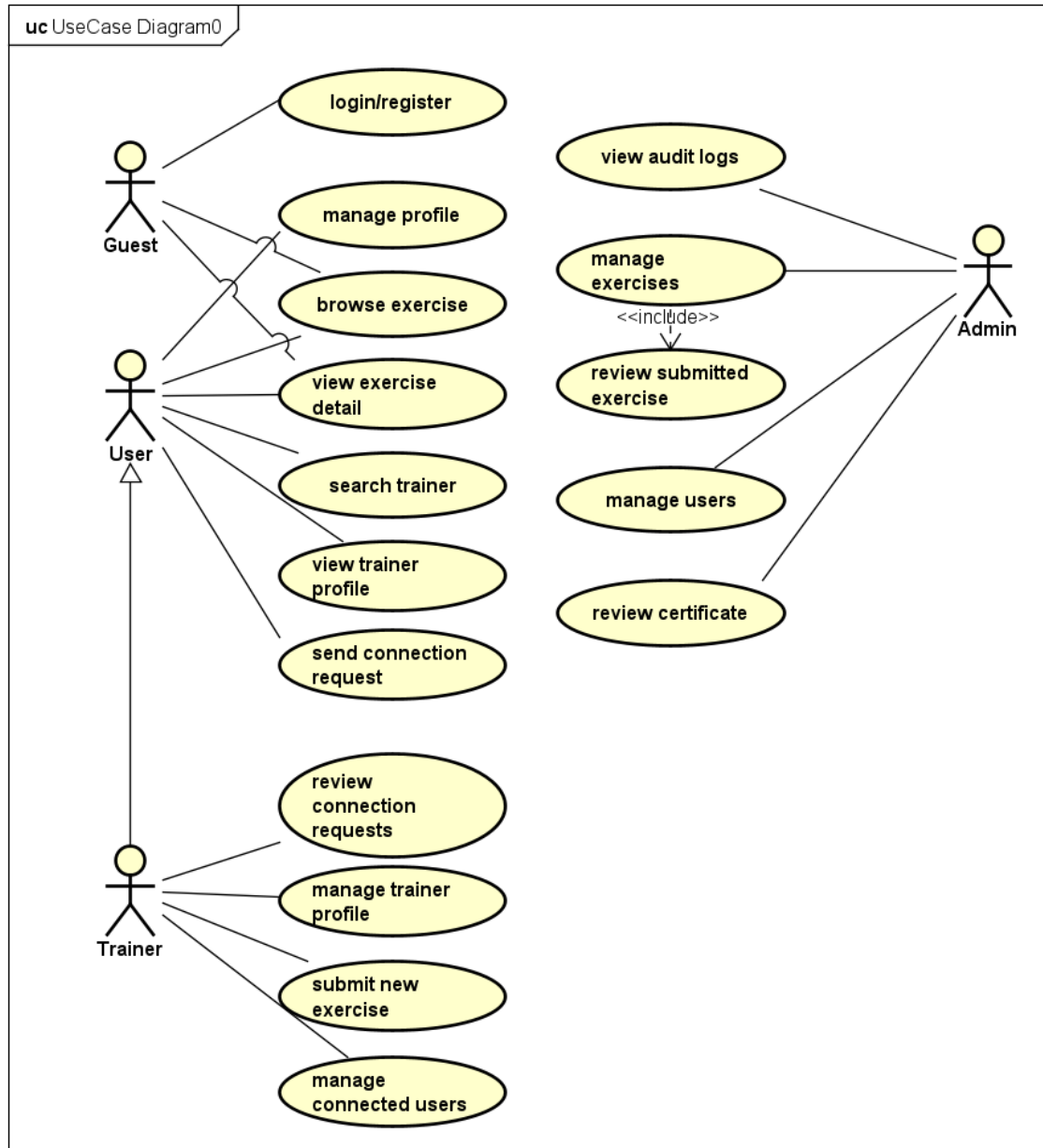
Rationale for the technologies used in the core demo:

- React + Vite supports reusable role-based pages and rapid development of the login, management, and exercise lookup interfaces.
- Express.js provides a lightweight REST API structure that maps naturally to authentication, users, exercises, taxonomy, and Admin modules.
- PostgreSQL is appropriate because users, roles, exercises, equipment, body parts, and muscles require relational integrity and many-to-many mappings.
- Zod validates request parameters and bodies before they reach the service layer, reducing inconsistent data and repeated manual checks.

Chapter 4 – FiFB Product

4.1 Functional Requirements

The system defines three stored roles: User, Trainer, and Admin. An unauthenticated visitor is not a database role and can only use public exercise pages and authentication pages. The table below lists only the functions evaluated in GR1.



[Figure 4.1: Use Case Diagram]

[Bảng 4.1: Functional Requirements by Actor]

Unauthenticated visitor (not a stored system role)

ID	Requirement	Description
G-01	Browse exercises	View active exercises with GIF, instructions, and taxonomy.
G-02	Filter exercises	Filter by body part, equipment, target muscle, secondary muscle, and keyword.
G-03	View exercise detail	View full exercise detail including all taxonomy associations.

G-04	Register account	Register as User or Trainer and create an account for later login.
G-05	Login / Logout	Authenticate with email and password and end the current session.

User (Authenticated, role = `user`)

ID	Requirement	Description
U-01	Manage profile	View and update personal profile fields.
U-02	Search exercises	Search active exercises by keyword.
U-03	Filter exercises	Filter by body part, equipment, target muscle, or secondary muscle.
U-04	View exercise detail	View GIF, instructions, body-part, muscle, and equipment information.
U-05	Use User interface	Access only routes and actions permitted for the User role.

Trainer (Authenticated, role = `trainer`)

ID	Requirement	Description
T-01	Manage trainer profile	Create/update bio, specialization, and years of experience.
T-02	Browse exercises	Search, filter, and view the shared exercise library.
T-03	Submit exercise	Submit new exercise information with pending status for Admin review.
T-04	Use Trainer interface	Access trainer-only profile and exercise-submission routes.

In the focused demo, the Trainer role does not manage users or exercise taxonomy.

Admin (Authenticated, role = `admin`)

ID	Requirement	Description
A-01	Manage users	List, filter, inspect, lock, unlock, or disable user accounts.
A-02	Review exercises	Approve, reject, or deactivate trainer-submitted exercises.
A-03	Create exercises	Create exercise records directly as Admin.
A-04	Update exercises	Edit exercise information and taxonomy mappings.
A-05	Manage equipment categories	List and add equipment or machine categories used to classify

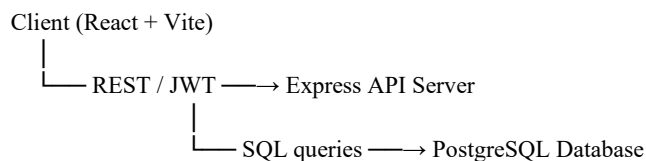
		exercises.
A-06	Manage other taxonomy	List and add body-part and muscle categories.
A-07	Use Admin interface	Access only routes protected for the Admin role.

4.2 Non-Functional Requirements

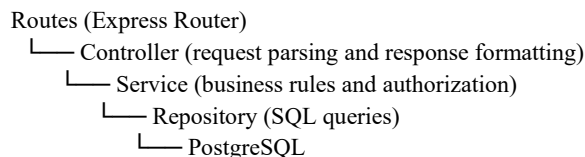
Category	Requirement
Performance	Standard login, CRUD, list, search, and filtering operations should respond within an acceptable time for a classroom demonstration; list endpoints use pagination.
Security	Passwords are hashed with bcrypt; JWT identifies the current role; protected routes enforce User, Trainer, or Admin permissions; CORS and validation are applied.
Data integrity	UUID primary keys, foreign keys, unique constraints, valid role/status values, and normalized exercise-taxonomy relationships.
Reliability	Centralized backend error handling and frontend error states prevent a failed request from breaking the entire demonstration flow.
Maintainability	The backend separates routes, controllers, services, repositories, and validation; the frontend separates API clients, routes, layouts, and role pages.
Usability	Role-specific navigation and clear search/filter controls allow each actor to complete the required demo tasks with minimal steps.

4.3 System Architecture

The evaluated GR1 architecture contains three main parts:



The backend follows a layered modular pattern:



Each core business domain is organized with the following responsibilities:

- ``.routes.js`` — Express route definitions with middleware composition.
- ``.controller.js`` — Thin controllers delegating to services.
- ``.service.js`` — Business logic and validation.
- ``.repository.js`` — SQL queries and database interaction.

- The modules directly evaluated in this report are `auth`, `users`, `trainers`, `exercises`, `exerciseTaxonomy`, and `admin`. Other repository modules are implemented extensions and are outside the focused GR1 evaluation.

[Figure 4.3: Entity–Relationship Diagram]

Core Group 1: Users, Roles, and Authentication

Core Group 2: Trainer Role Extension

Table	Description
`trainer_profiles`	A one-to-one extension of `app_users` for the Trainer role, containing basic biography, specialization, and experience information.

Core Group 3: Exercises, Equipment, and Classification

Table	Description
`exercises`	Stores exercise name, GIF URL, instruction steps, source, lifecycle status, and imported source data. Admin can create/update records and review Trainer submissions.
`body_parts`	Taxonomy: body parts (e.g., chest, back, legs).
`equipments`	Stores equipment or machine categories used to classify exercises, such as barbell, dumbbell, cable, or machine.
`muscles`	Taxonomy: muscle groups (e.g., pectorals, biceps, quadriceps).
`exercise_body_parts`	Many-to-many: exercises ↔ body parts.
`exercise_equipments`	Many-to-many: exercises ↔ equipment.
`exercise_muscles`	Many-to-many with role: exercises ↔ muscles (target or secondary).

Supporting Group: Administrative Traceability

Table	Description
`audit_logs`	Records sensitive Admin changes such as user-status and exercise-status updates. It supports traceability but is not a separate user-facing GR1 function.

Tables for favorites, workout planning, workout tracking, trainer connections, certificates, reviews, notifications, and chat remain in the full schema but are not analyzed as GR1 requirements.

4.5 Backend Implementation

4.5.1 API Endpoints

The focused backend exposes the following core endpoint groups:

[Bảng 4.3: API Endpoints Summary]

Resource	Endpoints	Description
Auth	9 endpoints	Register, verify email, login, refresh, logout, current user, change password, request reset, and reset password. Login is the principal GR1 flow.
Users	5 endpoints	Get/update own profile; Admin lists users, views a user, and changes account status.
Trainers	Core profile and submission endpoints	Manage the Trainer profile and submit a pending exercise for Admin review.
Exercises	7 endpoints	User lists/views active exercises; Admin creates, updates, reviews, rejects, and deactivates exercises.
Exercise Taxonomy	6 endpoints	List body parts, equipment, and muscles; Admin creates taxonomy entries, including equipment categories.
Admin	Core management endpoints	Manage user status and exercise review through Admin-protected routes.
Health	1 endpoint	Health check returning `{ status: "ok" }`.

4.5.2 Middleware Pipeline

For the core GR1 requests, the important middleware steps are:

1. Helmet: adds common security-related HTTP headers.
2. CORS: restricts requests to configured frontend origins.
3. JSON parser and Zod validation: parse and validate request data before business logic executes.
4. Authentication and role checks: validate JWT claims and restrict Admin- or Trainer-only routes.
5. Not-found handling: returns a consistent response for unknown routes.
6. Central error handling: converts validation, authorization, and database errors into a consistent API format.

4.5.3 Exercise and Equipment Data Management

The repository includes an import script and management endpoints that establish the exercise library used in the demo:

- The import script reads exercise records from ExerciseDB in pages.
- Exercises are inserted or updated by external identifier to avoid duplicate imported records.
- Body parts, equipment categories, and muscles are created and associated with exercises during import.
- Admin can list and add equipment categories through the exercise-taxonomy API and can change an exercise's equipment mappings when updating the exercise.
- User can then filter the exercise library by the equipment category stored in these relationships.

4.5.4 Core Role and Authorization Flow

The main backend flow can be summarized as follows:

- Login validates the email and password, then returns tokens containing the user role.
- Protected middleware reads the token and rejects requests made by an unauthenticated or unauthorized role.
- Admin endpoints call user services to list accounts and update account status.
- Admin endpoints call exercise and taxonomy services to manage exercises and equipment categories.
- Trainer endpoints allow profile maintenance and creation of pending exercise submissions.
- User-facing endpoints return only active exercises and support keyword and taxonomy filters.
- Repositories execute parameterized SQL queries and return normalized data to the service layer.

4.6 Frontend Implementation

4.6.1 Application Structure

For the focused GR1 demo, the relevant frontend structure is:

```
client/src/
├── api/           # authentication, users, trainers, exercises, taxonomy
├── components/    # forms, tables, pagination, status and error states
├── contexts/      # authenticated user and role state
├── layouts/       # authentication, User, Trainer, and Admin layouts
├── pages/
│   ├── auth/     # login and account pages
│   ├── user/     # exercise library, detail, and profile
│   ├── trainer/  # trainer profile and exercise submission
│   └── admin/    # user, exercise, and taxonomy management
└── routes/       # protected and role-based routes
```

4.6.2 Routing and Access Control

React Router separates the interface by authentication state and role:

- Authentication pages: login, registration, verification, and password recovery.
- User pages: profile, exercise library, exercise filters, and exercise detail.
- Trainer pages: trainer profile and new-exercise submission.
- Admin pages: user management, exercise management, and taxonomy management.

`ProtectedRoute` redirects unauthenticated users to login, while `RoleRoute` prevents a User or Trainer from opening Admin pages and prevents a User from opening Trainer-only pages.

4.6.3 HTTP Client and Token Management

The Axios-based HTTP client supports the core authenticated flows through:

- Automatic access-token injection for protected API requests.
- Automatic refresh attempts when the access token expires, followed by a retry of the original request.
- Session cleanup and redirection to login when token refresh fails.
- Consistent error objects for validation, permission, and server errors.
- Public GET requests for the exercise library and taxonomy when authentication is not required.

4.6.4 Role State Management

Global frontend state is intentionally limited for the core demonstration:

- `AuthContext` stores the authenticated account, supports login/logout, reloads the current user, and provides role-checking helpers.
- Route components and layouts use the current role to display only the navigation and actions assigned to User, Trainer, or Admin.

Local React state and reusable asynchronous hooks manage form input, filters, loading, errors, and API results.

4.6.5 Shared Components Used in the Core Demo

The following reusable components support the login, list, form, and management pages:

Component	Purpose
`ConfirmDialog`	Modal confirmation dialog for destructive actions.
`DataTable`	Generic data table with sortable columns.
`EmptyState`	Placeholder displayed when no data is available.
`ErrorBoundary`	Catches and displays React component errors.

`ErrorState`	Standardized error display with retry option.
`FormField`	Reusable form field wrapper with label and validation message.
`LoadingState`	Loading spinner/skeleton display.
`PageHeader`	Consistent page header with title and optional actions.
`Pagination`	Page navigation controls.
`StatusBadge`	Colored status indicator badge.

4.7 Role-Based Demo Workflows

4.7.1 Core Actions by Role

The following table connects each actor to the principal API flow used in the GR1 demonstration:

[Bảng 4.4: Core Role Workflows]

Role	Action	Main endpoint or flow
User	Login and load profile	`POST /auth/login` → `GET /users/me`
User	Search and filter exercises	`GET /exercises` with keyword and taxonomy parameters
User	View exercise detail	`GET /exercises/:id`
Trainer	Update trainer profile	`PUT /trainers/me/profile`
Trainer	Submit exercise for review	`POST /trainers/me/exercises`
Admin	Manage user status	`GET /users` and `PATCH /users/:id/status`
Admin	Manage exercises and equipment	Exercise CRUD/review endpoints and `/exercise-taxonomy/equipments`

4.7.2 User Exercise Lookup Flow

The most important end-user flow is exercise lookup:

- The User opens the exercise library and selects a keyword or one or more filters such as body part, target muscle, or equipment.
- The frontend sends a `GET /api/exercises` request. The backend queries active exercises and joins the relevant taxonomy tables before applying pagination.

The frontend displays matching exercise cards. Selecting a card opens the detail page with the exercise GIF, instructions, muscles, body parts, and equipment information.

4.8 Scope Boundary for Additional Implementations

4.8.1 Implemented Extensions Not Evaluated in GR1

The source repository contains functionality beyond the minimum assignment:

1. Favorite exercises, workout plans, workout sessions, statistics, streaks, and progression tracking.
 2. Trainer certificates, trainer–user connections, trainer reviews, chat, and notifications.
- Redis/Upstash caching and rate limiting, Docker support, CI workflows, and Render deployment configuration.
 - These features are retained in the codebase but are deliberately omitted from the core requirements, database discussion, and completion assessment in this report.
 - Their implementation does not compensate for, or replace, the required GR1 functions.

4.8.2 Reason for Exclusion

A focused report makes it easier to verify whether the system satisfies the supervisor’s stated GR1 objective rather than measuring the project by the number of optional features.

The evaluation therefore asks only whether the platform foundation, roles, database, login, user management, exercise management, equipment management, and User exercise lookup are demonstrated.

Optional modules are mentioned only to clarify that they exist in the repository and were intentionally removed from the detailed report scope.

4.8.3 Future Use of Existing Code

The additional modules can be reused in GR2 after the core requirements are reviewed and the missing physical-machine management functions are completed.

- Keeping them modular allows future work without changing the core authentication, user, exercise, and taxonomy modules.
- They should be presented as extensions rather than as mandatory GR1 deliverables.

Chapter 5 – Results and Evaluation

5.1 Completion Status

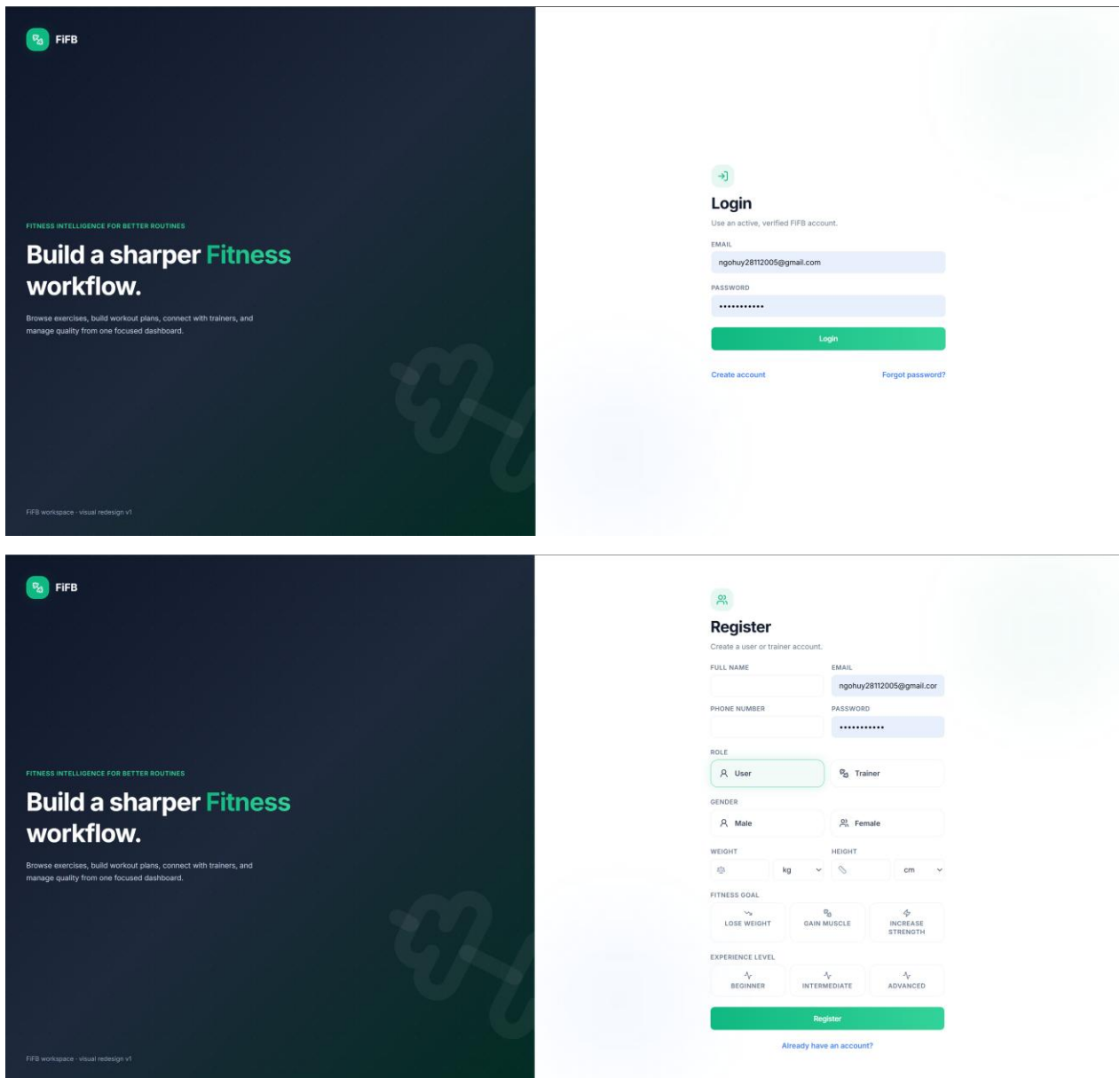
[Bảng 5.1: Feature Completion Status]

Feature Category	Status	Notes
Login and authentication	Completed	Email/password login, JWT access control, refresh, and logout are implemented.
Role definition and protected routes	Completed	The three roles are enforced by backend and frontend

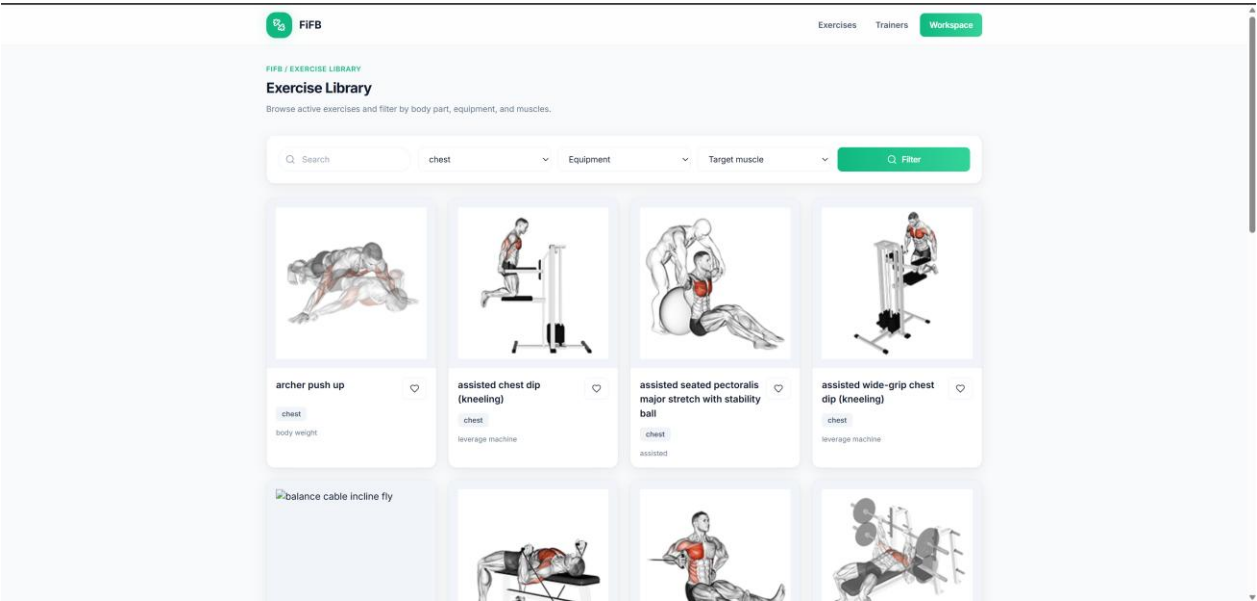
		guards.
Admin user management	Completed	Admin can list/filter users, inspect profiles, and change account status.
Admin exercise management	Completed	Admin can create, update, approve, reject, and deactivate exercises.
Equipment category management	Partially completed	Admin can list/add equipment taxonomy and associate it with exercises; this is not yet a full physical machine inventory module.
User exercise lookup	Completed	Keyword search, body-part, muscle, and equipment filters, pagination, and exercise detail are available.
Trainer basic role	Completed	Trainer profile management and exercise submission for Admin review are implemented.
Relational database design	Completed	The schema separates users, trainer profiles, exercises, equipment, body parts, muscles, and junction tables.
Core backend API	Completed	Authentication, users, trainers, exercises, taxonomy, and Admin endpoints are organized by module.
Core frontend demo	Completed	Login, role-based navigation, Admin management pages, Trainer submission, and User exercise pages are available.

5.2 Demonstration and Outputs

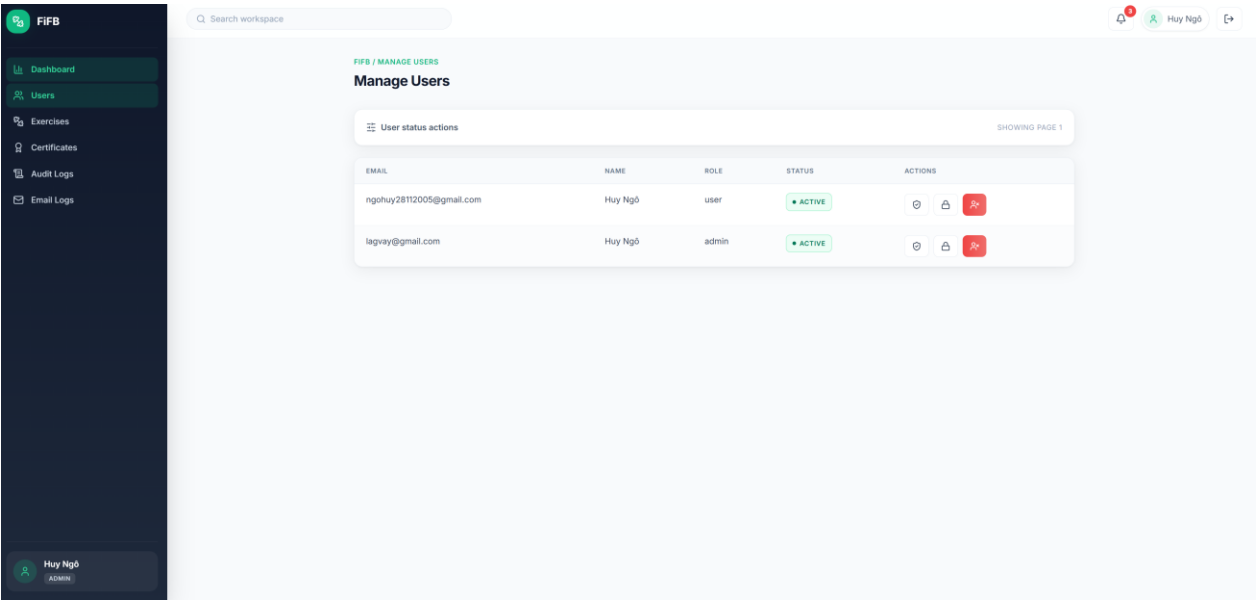
The following screenshots demonstrate the functions that are directly relevant to the focused GR1 scope:

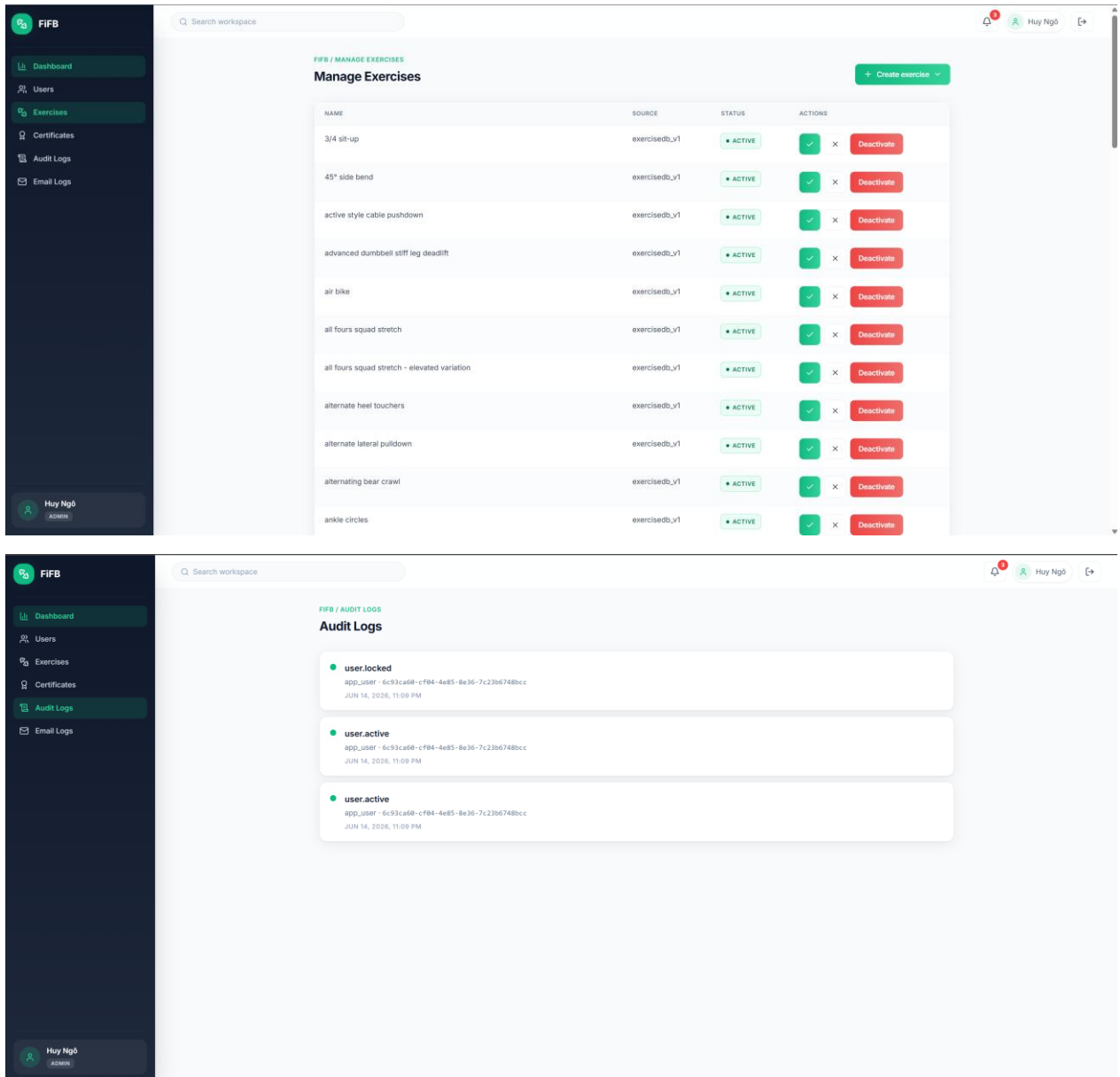


[Hình 5.1: Login and Registration Screens]



[Hình 5.2: User Exercise Library with Search and Filters]





[Hình 5.3: Admin User and Exercise Management Screens. The repository also contains an audit-log screen, but it is not evaluated as a core GRI requirement.]

5.3 Functional Evaluation

5.3.1 Authentication and Role Access

- Users can log in with email and password and receive an authenticated session.
- The backend token contains the account role and status, allowing middleware to reject unauthorized requests.
- The frontend redirects unauthenticated visitors to login and restricts User, Trainer, and Admin pages according to role.
- Passwords are stored as bcrypt hashes, and API request bodies are validated before use.

5.3.2 User Management

- Admin can retrieve a paginated and filtered user list.
- Admin can view account information and change status to active, locked, or disabled.
- Authenticated users can view and update their own basic profile fields.

5.3.3 Exercise and Equipment Management

- Admin exercise creation, update, review, rejection, and deactivation flows are implemented.
- Equipment categories are stored separately and connected to exercises through the `exercise equipments` junction table.
- The current implementation supports listing and adding equipment taxonomy entries and changing exercise mappings, but it does not manage individual physical machines, maintenance status, location, or availability.

5.3.4 User Exercise Lookup

- Users can search exercises by keyword and combine filters for body part, equipment, target muscle, and secondary muscle.
- Only active exercises are returned to normal users, while the detail page displays the complete exercise instructions and taxonomy associations.
- Pagination prevents the full library from being loaded in one response and supports a practical browsing demo.

5.3.5 Scope Assessment

- The repository demonstrates the required platform foundation, three roles, database design, login, user management, exercise management, and User exercise lookup.
- Equipment management is satisfied only at the category/taxonomy level. A strict interpretation of “machine management” as physical gym-machine inventory is therefore only partially satisfied.
- Additional workout, trainer-connection, chat, notification, caching, and deployment modules are not counted when assessing GR1 completion.
- The modular structure is adequate for extending the incomplete machine-management scope in GR2.

5.4 Limitations

The focused evaluation identifies the following limitations:

1. Equipment scope: the `equipments` table represents categories used by exercises, not individual gym machines with model, serial number, location, condition, or maintenance information.

2. Incomplete equipment CRUD: the documented taxonomy API supports listing and creating equipment entries, but dedicated update, deactivate, and delete operations for equipment categories are not clearly exposed.
3. Basic Trainer scope: the core report uses only Trainer profile and exercise submission; broader certificate, connection, review, and chat functions are outside the assignment scope.
4. Diagram scope mismatch: some existing figures were generated from the full repository and therefore show extension use cases or tables that are intentionally excluded from the focused evaluation.
5. Testing depth: unit tests exist, but complete end-to-end tests for every User, Trainer, and Admin flow have not been demonstrated in this report.
6. External exercise data: imported records depend on the quality and availability of the ExerciseDB source and may require manual review by Admin.

5.5 Development Plan for GR2

The next phase should first close the remaining gaps in the foundation before emphasizing advanced features:

Feature	Description
Physical Machine Management	Add a dedicated machine table with machine code, name/model, equipment category, gym location, operational status, purchase date, and maintenance information.
Full Equipment CRUD	Add update, deactivate, and delete rules for equipment categories with checks that protect existing exercise relationships.
Trainer Workflow Refinement	Clarify the Trainer dashboard and exercise-submission lifecycle without making trainer–client marketplace functions mandatory.
Focused Diagrams	Create a new use-case diagram and a core ERD that show only User, Trainer, Admin, users, exercises, equipment, and taxonomy relationships.
Comprehensive Testing	Add integration and end-to-end tests for login, role restrictions, user status changes, exercise CRUD, equipment management, and User search/filter flows.
Responsive UI	Systematically test and improve the core pages on mobile and tablet screen sizes.
Vietnamese Interface	Add Vietnamese labels and messages so the demo is easier to use in the target context.
Optional Extensions	Only after the foundation is accepted, selectively reintroduce workout tracking, trainer connections, chat, notifications, or AI features as GR2 extensions.

References

1. Express.js Documentation. (n.d.). Retrieved from <https://expressjs.com/>
2. React Documentation. (n.d.). Retrieved from <https://react.dev/>
3. PostgreSQL Documentation. (n.d.). Retrieved from <https://www.postgresql.org/docs/>
4. Vite Documentation. (n.d.). Retrieved from <https://vitejs.dev/>

5. Tailwind CSS Documentation. (n.d.). Retrieved from <https://tailwindcss.com/docs>
6. JSON Web Tokens (JWT) - RFC 7519. (2015). Retrieved from <https://datatracker.ietf.org/doc/html/rfc7519>
7. ExerciseDB API. (n.d.). Retrieved from <https://exercisedb.io/>
8. Zod Documentation. (n.d.). Retrieved from <https://zod.dev/>
9. Axios Documentation. (n.d.). Retrieved from <https://axios-http.com/docs/intro>
10. React Router Documentation. (n.d.). Retrieved from <https://reactrouter.com/>

FiFB Project Repository. (2026). Graduation-Research-FiFB. GitHub:
<https://github.com/ngohuyhust/Graduation-Research-FiFB>

Appendices

Appendix A: API Endpoint Reference

The complete API specification is documented in `Document/API.md`. For the focused GR1 scope, the relevant sections are:

- Authentication and token endpoints.
- User and role permissions.
- Trainer profile and exercise-submission endpoints.
- Exercise, taxonomy, and Admin management endpoints.

Appendix B: Database Schema

The complete SQL schema is available in:

- `Document/DatabaseSQL-v2.txt` — full schema; the report analyzes only the core user, trainer-profile, exercise, equipment, body-part, muscle, junction, and audit tables.

Appendix C: Focused Project Repository Structure

Graduation-Research-FiFB/

```

|— Document/
|   |— API.md           # API and role documentation
|   |— DatabaseSQL-v2.txt # Complete database schema
|— client/src/
|   |— api/             # Core API clients
|   |— pages/auth/      # Login and account pages
|   |— pages/user/      # Exercise lookup and detail
|   |— pages/trainer/   # Trainer profile and submission
|   |— pages/admin/     # User, exercise, and taxonomy management
|   |— routes/          # Protected and role-based routing

```

```
└─ server/src/
  └─ middlewares/      # Authentication, roles, validation, errors
  └─ modules/auth/     # Login and token logic
  └─ modules/users/    # User profile and Admin user management
  └─ modules/trainers/ # Basic Trainer functions
  └─ modules/exercises/ # Exercise query and management
  └─ modules/exerciseTaxonomy/ # Body parts, muscles, equipment
  └─ modules/admin/    # Admin review operations
```

Other modules remain in the repository as extensions but are outside the focused GR1 report scope.